Scalable Parallel Numerical Integration of Decoupled Delay Differential Equations

Nick Twum
dept. name of organization
name of organization
City, Country
email or ORCID

Max Fleming
dept. name of organization
name of organization
City, Country
email or ORCID

John A. Stratton
dept. name of organization
name of organization
City, Country
email or ORCID

***Abstract*—Delay Differential Equations are a valuable modeling technique in multiple domains, but have not been well supported by scalable numerical solvers. In this paper, we present DDEINT, a parallel numerical solver for delay differential equations, and summarize the core data structures needed to preserve and reference the past simulation results referenced by current rate functions. We demonstrate how classical numerical integration library interfaces limited parallel scalability, overcome by providing alternative user-friendly interfaces that expose a higher level of data parallelism to the runtime library. Furthermore, we observe that when domains in large-scale DDE systems interact only through delayed effects, either naturally or through intentional mathematical substitutions, it is possible to partially decouple the simulation of those domains and dramatically increase the scalability of their simulation. We test DDEINT on shared and distributed memory systems, simulating models drawn from epidemiology and systems biology, showing substantial speedups from decoupled simulation methods.**

***Index Terms*—Delay differential equations, numerical integration, parallel computing, decoupled simulation.**

I. Introduction

Delay differential equations (DDEs) are important models for various problems that include processes with a mixture of slow and fast processes. For instance, in epidemiology, infection transmission usually happens at a very small latency compared to infection incubation. The transition from "susceptible" to "infected" can be modeled as instantaneous, while the transition from "infected" to "contagious" may best be described as a delayed effect [**?**].

We wanted to build a library to enable models of this kind to be effectively simulated at scale. However, the traditional interface of numerical integration libraries like ODEPACK force the user to implement the parallelism in their own code within the function to be numerically integrated, as there is little parallelism available within the numerical integration library itself.

Many models that require scalable compute come from multi-scale models where individual domains are modeled as DDEs and those domains have some coupling between each other, such as spatial discretizations of PDEs with delays or agent-based models of a population. As a result, many of the optimizations explored for PDE integration may be relevant [**?**], [**?**], [**?**] as a means of reducing the communication frequency between parallel compute units to improve scalability. However, multi-domain DDEs delays often specifically use those delays to model slow spatial interactions and exhibit the property that dependencies between domains are inherently relaxed by the delays in the model. This presents an opportunity to partially decouple the simulation of different domains without the redundant computation or predetermined dependency graphs required for typical techniques.

In this paper, we introduce DDEINT, a scalable numerical library for solving initial value problems of delay differential equations. Section II introduces the core numerical engine, an efficient C++ implementation of prior work extending the Dormand-Prince numerical method to support delays. Section III shows how we extended this basic method with modern C++ features to make the user configuration of numerical integration and parallel execution methods possible through simple parameters. Section IV introduces our further extensions to enable multi-domain simulation construction and expose the resulting data parallelism to the library itself. In Section V we demonstrate how such simulations, when the domains have only delayed interactions with each other, can be simulated with partially decoupled parallel execution units. We outline some notable implementation details of our shared-memory and MPI parallelizations of the library in Section **??**. Sections **??** and **??** describe our evaluation methodology and results, followed by a final summary of related work and concluding reflections in Sections **??** and **??**.

II. Background

A Delay Differential Equation (DDE) is a differential equation where by the rate of change of a system depends not only on the current state of a system, but also past state. A simple single-delay differential equation might be notated as:

$$\frac{d}{dt}x(t) = f(x(t), x(t-\tau))$$

Where τ is the delay in the system

Numerical integration of DDEs builds on the same methods as ODEs, with the primary addition of state history tracking so that it can be referenced when calculating future rates of

change. Since state values are sampled over time in numerical integration, when referencing the historical value of some variable, that value will need to be inferred by interpolation from the closest samples in time available. Our history data structure is a collection of ring buffers, each holding either a timestamp or a variable state (current value and current rate of change.) When a new state vector is calculated by the numerical integrator, the timestamp of the current step along with each variable's state is appended to their respective ring buffers. The user of the library is required to provide, for each variable, the maximum delay at which it might be accessed. When the current simulation time is more than that maximum delay ahead of the oldest entry in the ring buffer, that value is removed from the ring buffer to keep RAM requirements limited. We adapt the DDE integration method from FitzJohn and Hinsley [**?**] implementing the Dormand-Prince [**?**] numerical integration method and using a Hermite spline interpolation method for inferring historical state values.

There are a wide variety of DDEs based on the restrictions or properties of the delays themselves. Some systems have a fixed number of constant delays, while others may allow τ to be a function of time itself. Our library does not put any restrictions on what delays are allowed, except that the user must provide upper bounds of how large delays could become so that it can know when simulation history will no longer directly affect current rates of change and can be safely discarded, keeping memory requirements under control.

III. Library Design

DDEINT separates the specification of model equations from the organization of numerical integration and execution. The user supplies a function that evaluates the derivatives of one cell, together with model and solver parameters. The library constructs the corresponding state and history views and invokes the function according to the selected simulation policy. This interface exposes cell-level work to the library without requiring the model function to implement parallel scheduling.

A. Cell Function and History Access

A cell represents an individual subsystem, such as a spatial domain or an agent. Its state is partitioned into private variables, whose histories are used within the cell, and public variables, whose histories may be accessed by other cells. Public denotes availability for inter-cell history access rather than visibility in the C++ access-control sense.

The user defines the cell's rate equations through a callback with the following signature:

```
void cell_func(
    double t, Cell& cell,
    CellHistory& history);
```

Here, t is the evaluation time, which may be an intermediate Runge–Kutta stage time. The `Cell` argument is a non-owning view of the cell's entries in the state and derivative arrays. The callback reads the supplied state and writes the corresponding derivatives; it does not advance the cell's integration time or select a step size. During concurrent evaluations, the callback must preserve the supplied state and shared model data and restrict derivative writes to its own cell.

The `CellHistory` argument provides access to historical values without exposing their storage layout. Its `local_at` operation retrieves a private variable's past value from the current cell, while `global_at` retrieves a public variable's past value from a specified cell. The same object provides access to per-cell coefficients and user-supplied model context, such as interaction topology.

This history interface is shared by the cell-based simulation policies. In CellArray execution, private and public histories are stored together in the full-state history. In decoupled execution, private histories are maintained per block, while public histories are exposed through the global history. The cell function therefore uses the same lookup interface under either storage organization.

B. Policy-Based Configuration

The library exposes three compile-time configuration choices: the simulation policy, execution policy, and integration policy.

- **Simulation policy:** `CellArrayDDE` combines all cells into one state vector integrated by a single solver. `DecoupledSim` partitions the cells into blocks and assigns a separate solver to each block. Both accept the same cell-function signature. The library also provides `DDESim`, a whole-state interface in which the user supplies a callback over the complete state and derivative vectors.
- **Execution policy:** A Boolean template parameter selects sequential or TBB-parallel integration dispatch using `if constexpr`. In `CellArrayDDE`, this choice controls the loop over cells within each derivative evaluation. In `DecoupledSim`, it controls the loop over independently integrated blocks. Other operations, including history publication and MPI payload packing, may also use TBB.
- **Integration policy:** A template-template parameter supplies the numerical integrator, independently of the execution flag. The current implementation provides `DoPri_5`, an adaptive Dormand–Prince method. An alternative integrator must satisfy the solver interface used by the selected simulation policy.

The cell callback is supplied as a function-pointer template argument, making its identity available at compile time. This design permits compiler specialization and, where appropriate, inlining of the callback into derivative evaluation. Similarly, compile-time execution selection avoids a runtime branch between the two integration-dispatch paths. These properties do not eliminate scheduling, history-access, or communication costs.

C. Simulation Construction and Parameters

The `Simulation` class accepts an instantiated simulator type and packages three parameter objects.

`SimStructParams` describes the model structure, including the cell count, private/public variable counts, block size, history-retention bounds, prehistory functions, and coefficients. `SimSolverParams` contains initial states, step-size settings, and error tolerances. `SimRuntimeParams` specifies the simulation interval, output sampling, and synchronization-window configuration.

For example, a multithreaded CellArray simulation is constructed as:

```
using Policy =
    CellArrayDDE<cell_func, double,
                 Parallel>;
Simulation<Policy, double> sim(
    structure, solver, runtime);
sim.initialize();
auto results = sim.run();
```

The integration policy defaults to `DoPri_5`. Replacing `CellArrayDDE` with `DecoupledSim` selects block-based integration while retaining the cell callback. Decoupled execution additionally requires an appropriate block size and a synchronization schedule consistent with the cross-block delays, as described in Section V.

IV. Supporting Multi-Domain DDEs

Many DDE models consist of interacting subsystems, such as spatial domains or individual agents. The cell interface introduced in Section III allows the user to express the rate equations of one subsystem. The CellArray simulation policy applies this function across the model's cells, assembling their derivatives into the whole-state vector required by the numerical integrator. This organization exposes parallel work to the library while retaining a single solver for the complete system.

A. CellArray Representation

The user defines the decomposition of the model into cells; CellArray does not perform spatial discretization itself. Cells may represent the domains of an already discretized spatial model or subsystems defined directly by the application. The current interface uses a common number of private and public variables per cell, while per-cell coefficients and model context can describe heterogeneous parameters and interactions.

For a model containing N cells, let $\mathbf{u}_i(t)$ denote the state of cell i. CellArray represents the complete state as

$$\mathbf{U}(t) = \left[\mathbf{u}_0(t)^\mathsf{T}, \ldots, \mathbf{u}_{N-1}(t)^\mathsf{T}\right]^\mathsf{T}. \tag{1}$$

The state and derivative vectors are stored in separate contiguous arrays with a cell-major layout. Each cell occupies a consecutive slice, with its private variables followed by its public variables. Fig. 1 illustrates this organization.

At each derivative evaluation, the library constructs a `Cell` view of each cell's state and derivative slices, together with a `CellHistory` view for historical lookups. The user-defined cell function reads the supplied state and historical values and writes the derivatives for that cell. Delayed lookups of other cells' public variables express inter-cell coupling, so disjoint state slices do not imply that the modeled subsystems are uncoupled.

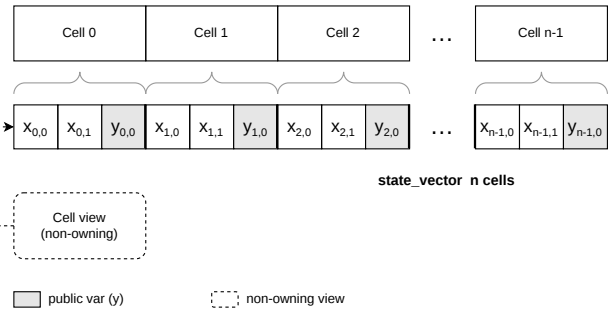


Fig. 1. Example CellArray state layout with two private variables ($x_{i,0}$ and $x_{i,1}$) and one public variable ($y_{i,0}$) per cell. Cells occupy consecutive slices of the state array. A non-owning cell view references one slice without copying its values; the derivative array uses the same indexing.

The CellArray adapter translates these per-cell evaluations into the whole-state callback expected by the integrator. In contrast, the monolithic interface requires the user to implement that whole-state callback directly, including any cell indexing and parallel dispatch.

B. Parallel Derivative Evaluation

In multithreaded execution, CellArray invokes the cell function through a TBB parallel loop. Each invocation writes to a distinct derivative slice, while the supplied stage state and stored historical samples remain unchanged throughout the evaluation. Provided the callback follows this access contract and does not modify shared model data, cell evaluations can execute concurrently even when they read overlapping historical data.

The parallel loop completes before the solver consumes the resulting derivatives to construct the next integration stage. With Dormand–Prince integration, this dispatch occurs at each right-hand-side evaluation, including intermediate Runge–Kutta stages. The sequential execution policy evaluates the same cells through an ordinary loop.

CellArray therefore parallelizes derivative evaluation without giving cells independent integration trajectories. A single solver computes an error estimate over the complete state vector and makes one step-acceptance decision and step-size update for all cells. The approach retains synchronization at each derivative evaluation and shared adaptive stepping across the system. These constraints motivate the block-based decoupled simulation described in Section V.

V. Decoupled Simulation

In CellArray, the variables of all cells form a single state vector integrated by one Dormand–Prince solver. Although derivative evaluations are parallelized across cells, all cells share the integration time, adaptive step size, and step-acceptance decision.

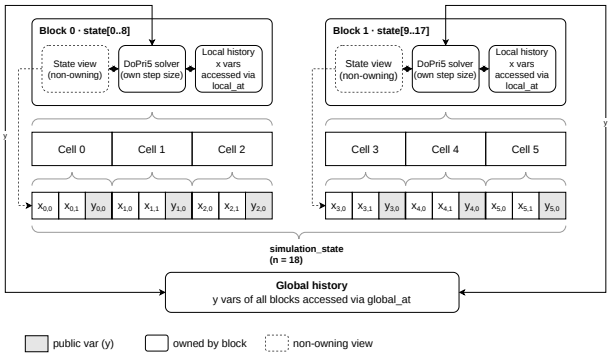


Fig. 2. The state vector is partitioned into blocks of consecutive cells. Each cell holds private variables (x, white) followed by public variables (y, shaded), and thick borders mark cell boundaries. Each block owns a DoPri5 solver and a local history, and accesses its slice of the state vector through a non-owning view (dashed). Private variables are stored in the block's local history (`local_at`). Public variables from all blocks are stored in a single shared global history (`global_at`), which is how cells in different blocks see each other's state.

The decoupled interface instead partitions the cells into blocks, each integrated by a separate solver. Each block maintains its own integration time, adaptive step size, and step-acceptance decision. Cells within a block continue to share these quantities. Private-variable histories are maintained locally, while public-variable histories provide the delayed values required for interactions between blocks.

A. Independent Adaptive Stepping

In CellArray, the solver computes an error estimate over the entire state vector. A rejected step therefore requires recomputation for all cells, including cells that might have satisfied a separate local acceptance test. When cells exhibit heterogeneous numerical behavior, the shared adaptive controller can also impose smaller steps on cells that could otherwise advance with less work.

Decoupling confines error estimation and step rejection to individual blocks. Blocks can consequently follow different adaptive step sequences, potentially reducing unnecessary integration work. This independence applies between blocks; cells within each block remain subject to a shared adaptive controller.

B. Delay-Bounded Independence

Independent advancement is possible when all cross-block interactions use delayed public states and those delays have a positive lower bound $\tau_{\min}$. Suppose the public histories of all blocks are available through simulation time T. For any cross-block delay $\tau \geq \tau_{\min}$, an evaluation at time t satisfies

$$T \leq t \leq T + \tau_{\min} \quad \Longrightarrow \quad t - \tau \leq T.$$

Thus, all required cross-block lookups fall within the already available history. Blocks can advance through this interval using different adaptive steps without requiring newly computed public states from other blocks. These lookups use the historical values specified by the model, rather than approximating current states with outdated values.

C. Windowed Synchronization

The simulation proceeds through synchronization windows. During each window, blocks advance independently, updating their private histories and buffering the public states, derivatives, and timestamps produced at accepted steps. The accumulated public histories are published at synchronization boundaries.

This organization allows blocks to perform multiple integration steps between shared-history updates, replacing CellArray's synchronization at each derivative evaluation with coarser-grained block execution. The window duration must ensure that every required delayed lookup remains within the available shared history.

Unequal block workloads can nevertheless leave workers idle while the remaining integration work in a window completes. Distributed execution also incurs history-exchange costs. By choosing windows shorter than the delay-based advancement limit, the execution pipeline can leave room for further integration while an exchange is in flight, provided the transferred history is published before it is needed. The following sections describe the shared-memory execution and MPI communication pipeline.

VI. Multithreaded Pipeline

The execution policy selects sequential or multithreaded integration at compile time. In CellArray, TBB parallelizes cell evaluations within each derivative callback. In the decoupled interface, TBB instead parallelizes integration across blocks, with each block's solver evaluating its cells sequentially.

Shared-memory execution alternates between block integration and history publication. During the integration phase, each block advances through the output times assigned to the current synchronization window using its own adaptive step sequence. After each accepted step, the block updates its private history and buffers its public states, derivatives, and timestamp. These history samples are retained independently of the requested output samples.

The block-integration parallel loop completes before history publication begins. A second parallel loop appends each block's buffered samples, in time order, to its associated compound ring buffer in the global history. Because different blocks update distinct ring buffers, publication can proceed concurrently. This loop completes before integration resumes, separating reads of shared historical samples from updates to those samples.

Blocks therefore perform multiple integration steps between shared-history updates, rather than synchronizing at each derivative evaluation as in CellArray. The synchronization schedule must respect the delay-based dependency bound established in the preceding section. TBB schedules block work across worker threads, although unequal workloads can still leave workers idle near the end of an integration phase.

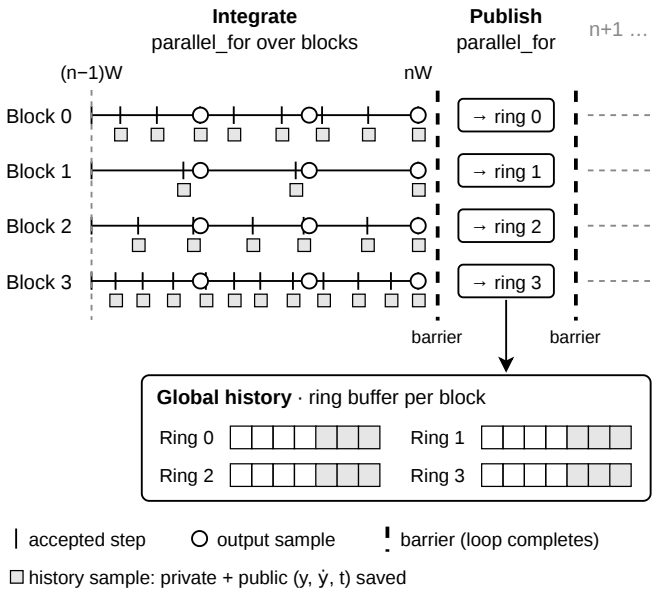


Fig. 3. Shared-memory execution of one synchronization window. In the integration phase, a parallel loop advances each block from $(n-1)W$ to nW with its own adaptive step sequence (ticks). After each accepted step, the block updates its private history and buffers its public states, derivatives, and timestamp (shaded squares). These history samples are independent of the requested output samples (circles). After a barrier, a second parallel loop appends each block's buffered samples, in time order, to that block's compound ring buffer in the global history. Because the blocks write to distinct ring buffers, publication proceeds concurrently. A second barrier separates publication from the next integration phase, so shared historical samples are never read and updated at the same time.

VII. MPI Hybrid Pipeline

We extend the shared-memory implementation with MPI by assigning a contiguous range of blocks to each rank. Within a rank, TBB schedules integration across the assigned blocks. Each rank maintains the private histories of its own blocks and a global history containing public variables from all blocks. This organization supports delayed cross-block interactions without requiring the user-defined cell function to perform MPI communication.

A straightforward distributed implementation would integrate all local blocks through a synchronization window, exchange their accumulated public histories, and publish the received samples before beginning the next window. This approach exposes both communication costs and waiting caused by unequal rank workloads. The delay-based dependency bound allows us to defer history publication and overlap the exchange with subsequent integration.

A. Variable-Sized History Exchange

Each rank packages the public-history samples produced by its blocks during a window. The payload contains block identifiers, sample counts, timestamps, public state values, and corresponding derivatives. Because blocks use independent adaptive step sequences, the number of accepted steps, and therefore the payload size, can vary across ranks and windows.

Before transferring these payloads, ranks exchange their byte counts using `MPI_Iallgather`. Once this size exchange completes, each rank computes the receive counts and displacements and allocates a receive buffer for the actual combined payload size. The public histories are then exchanged using `MPI_Iallgatherv`. An all-gather is used because a local cell function may request delayed public values from any block, including blocks owned by other ranks.

Exchanging actual sizes avoids allocating receive buffers for a pessimistic maximum number of integration steps. The variable-count transfer also avoids transmitting unused capacity, as would occur with a fixed-size padded representation.

B. Overlapping Communication and Integration

The pipeline transfers each window's public history while computing the following window. For window k, execution proceeds as follows:

1) After local integration completes, each rank packs the public-history samples from window k and initiates the payload-size exchange.
2) Before integrating window $k+1$, each rank completes the size exchange, prepares its receive buffer, and initiates the history transfer using `MPI_Iallgatherv`.
3) The rank integrates its blocks through window $k+1$ while the history transfer remains outstanding, using previously published global history.
4) Before integrating window $k+2$, the rank completes the transfer with `MPI_Wait` and publishes the received samples into the global history.

For uniform windows of duration W aligned with the integration targets, this publication schedule requires

$$2W \le \tau_{\min},$$

where $\tau_{\min}$ is a positive lower bound on every cross-block interaction delay. During window $k+1$, the published history covers the end of window $k-1$, while window k's history is being transferred. The latest evaluation time in window $k+1$ is therefore at most $2W$ beyond the published-history horizon. Under the stated bound, all delayed lookups remain within the available history. For nonuniform windows, the same dependency condition must hold using the actual integration endpoints and published-history horizon.

Two alternating send buffers preserve the payload of an outstanding transfer while the next window's payload is packed. After transfer completion, received block records are identified and their samples are appended in time order to the corresponding global-history buffers. Publication is parallelized across blocks because distinct blocks update distinct ring buffers. Publication completes before integration resumes.

Nonblocking communication provides an opportunity to hide transfer time behind integration; it does not guarantee that a transfer has finished when integration completes. Any remaining communication time is paid at the explicit completion boundary. The amount of latency hidden depends on the available integration work, rank imbalance, and the MPI implementation's communication progress.

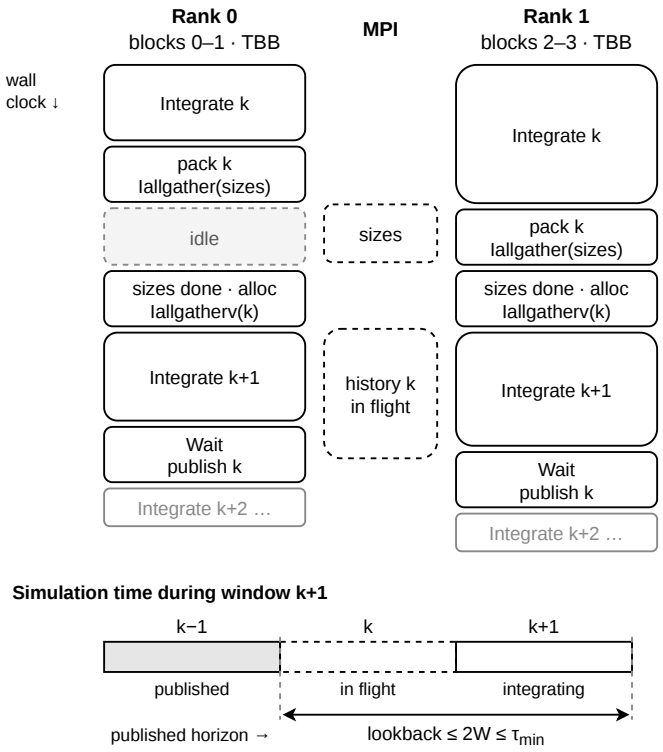


Fig. 4. Overlapped MPI history exchange for window k. Top: wall-clock execution on two ranks. After integrating window k, each rank packs its public-history samples and posts `MPI_Iallgather` to exchange payload sizes. A rank that finishes integration early idles until the size exchange completes. Each rank then allocates its receive buffer, posts `MPI_Iallgatherv`, and integrates window $k+1$ while window k's history is in flight (dashed). Any transfer time not hidden behind integration is paid at `MPI_Wait`, after which the received samples are published before window $k+2$ begins. Bottom: simulation-time view during window $k+1$. History is published through window $k-1$, so delayed lookups may reach back up to $2W$, which requires $2W \leq \tau_{\min}$.

VIII. Evaluation Methodology

To evalaute our test, we implemented three models in the CellArray and DecoupledBlocks Interfaces.

- SEIR: An epidemiological simulation with 4 private variables (susceptible population, exposed population, infected population, and recovered population) The global variable /(I_{pub}) mediates
- TIV: Agent based simulation with cell representing an individual. T (healthy target cells), I (Infected cells), E (Immume effector cells) (killer t-cells) (V): free virions.
- SOGEN:

Explain our test models, test computing environments, etc.

IX. Evaluation Results

Point 1: the CellArray and DecoupledCellArray interfaces result in fewer and simpler lines of code that must be written by the user. Results: Lines of Code for a multithreaded version of our models using the direct and CellArray interfaces.

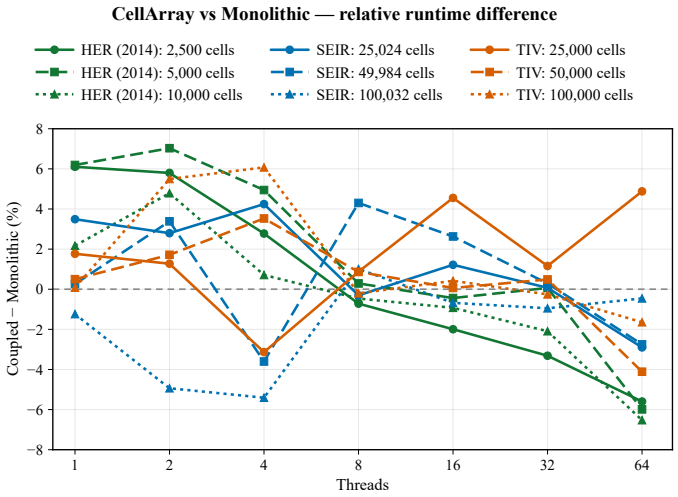


Fig. 5. Relative runtime difference between the CellArray, and Monolithic lockstep implementations for SEIR, TIV, and HER at different cell sweeps. The two appraoches differ by less than 7% with no visible trend indicating performance difference of one over the other.

A. The CellArray version does not sacrifice performance to achieve its improved usability over the Monolithic version. Results: multithreaded weak and strong scaling of models of both on the threadripper.

The CellArray interface does not sacrifice performance to obtain its usability gains over the monolithic interface. Across all 63 strong-scaling configurations (three models, three problem sizes, and 1–64 threads on the Threadripper node), CellArray runtime is within 7.03% of the monolithic implementation, with a median absolute difference of 1.99%. The two interfaces also perform identical numerical work: integrator calls, accepted solver steps, cell evaluations, and the final simulation state are bit-identical in every configuration, and peak memory use differs by at most 2.3%. The small runtime differences that remain are systematic rather than random. CellArray carries a modest serial overhead at low thread counts, at most 6.2% at one thread for HER, which we attribute to the library-managed construction of cell views on each step. This overhead is amortized as thread count grows: at 64 threads CellArray is faster than the monolithic version in eight of nine model/size combinations, by up to 6.5%. The same pattern appears under weak scaling. There, the single-thread overhead is more visible (7.8–15.5%) because the single-thread runs are short (0.6–6.3 s) and the absolute gap is only 0.09–0.53 s, but it disappears at 16 threads and above. Because both interfaces share the same lockstep integration scheme, they also share the same scaling limits: 64-thread strong-scaling speedups range from 5.1× to 18.1× for both. We therefore conclude that moving parallel dispatch and view construction from user code into the library costs no meaningful performance, and that the poor scalability of both lockstep variants comes from the integration scheme itself rather than from the interface.

B. Point 3: Decoupled simulation enables substantially better performance and scalability. Results: multithreading weak and strong scaling of CellArray and Decoupled Simulation (and monolithic? This might be one graph including the data from Point 2 as well.)

Decoupled simulation delivers substantially better performance and scalability than either lockstep approach (Fig. 6). It is faster than both CellArray and Monolithic in every one of the 54 strong- and weak-scaling configurations we measured. At 64 threads, decoupled simulation achieves strong-scaling speedups of 31.6–39.4× for SEIR and HER (49–62% parallel efficiency) and 18.8–20.2× for TIV, compared with 5.1–18.1× for the lockstep variants. In absolute terms, decoupled runs at 64 threads are 3.1–7.7× faster than the best lockstep runs at the same thread count, and 32–75× faster than lockstep on a single thread. The lockstep approaches also show sharply diminishing returns: doubling from 32 to 64 threads reduces their runtime by only 6–30%, compared with 24–56% for decoupled simulation. Weak scaling shows the same gap, with 64-thread efficiencies of 32–54% for decoupled simulation against 4–14% for lockstep. The work counters show that two separate mechanisms produce this advantage, and the models exercise them to different degrees. In HER, whose cells are homogeneous, both approaches perform exactly the same amount of integration work: blocks take an average of 5387 steps against 5388 global lockstep steps. Decoupled simulation is nonetheless 6.0–7.7× faster at 64 threads, so this gain comes almost entirely from parallel efficiency rather than from saved work. Decoupled blocks synchronize only at the 66 sync windows set by the minimum coupling delay, instead of after each of roughly 5400 global steps, and each block's working set stays small and cache-resident. In TIV, where cell dynamics are heterogeneous, per-block step-size control additionally avoids the global error-driven step reductions and rejections described in Section V. As a result, decoupled TIV performs 2.2–2.5× fewer cell steps than lockstep and is 2.9–3.7× faster even on a single thread. SEIR falls between these two cases, combining a 1.4–1.5× single-thread advantage with a further 2.2× advantage in scaling. Decoupling therefore improves scalability even when it saves no arithmetic, and it compounds that improvement with algorithmic savings in models where cells evolve at different rates.

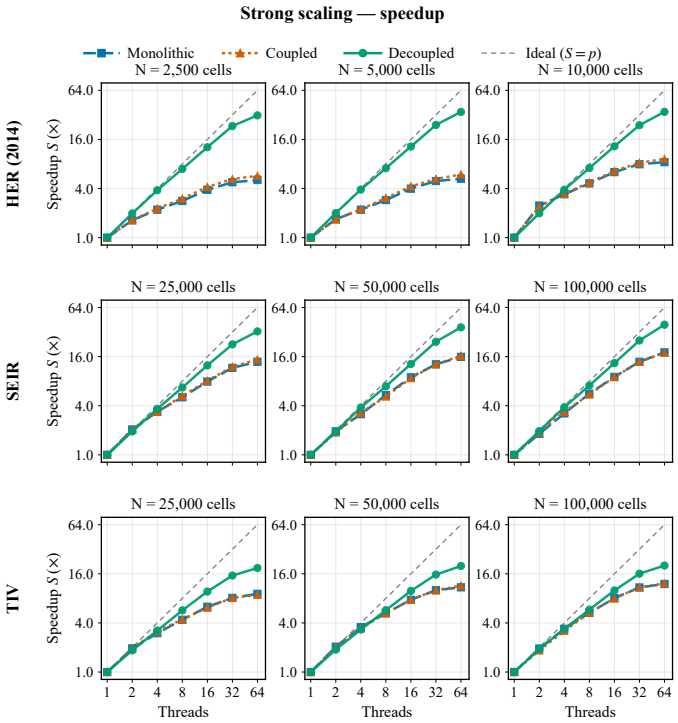


Fig. 6. Strong-scaling speedup versus thread count for each model. Decoupled scales far better than monolithic and lockstep for all three models. Approximately 20-39x gain in performance on 64 threads versus about 8-18x for Monolithic and Coupled.

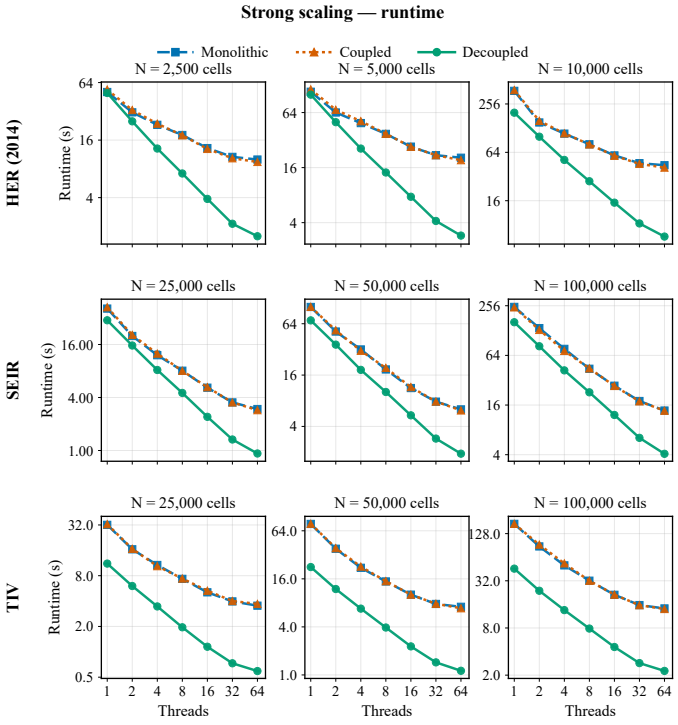


Fig. 7. Strong-scaling runtime versus thread count for each model at various problem size. Both axes in $\log_2$ scale.

C. Distributed Scaling and Communication Overlap

Asynchronous overlap of the MPI history exchange provides an incremental runtime improvement, but does not remove the communication costs that limit distributed scalability. We evaluated decoupled SEIR and TIV on a 32-node AWS cluster using m6a.2xlarge instances, with four physical cores, one MPI rank, and four threads per node, communicating over a TCP interconnect.

Under strong scaling, decoupled SEIR remains near ideal through eight ranks, with 93–104% parallel efficiency. At 32 ranks, efficiency remains at 51–60%, reaching a 19.3× speedup for one million cells. Larger problems retain higher efficiency because communication costs are amortized over more computation. Relative to immediate-wait execution of

the same history-exchange pipeline, overlap reduces runtime by 0–3.3% at two to four ranks and by 2.7–11.8% at 32 ranks.

The communication counters help explain the diminishing efficiency. The all-gather exchange replicates public history across ranks, so each rank receives history covering the complete system rather than only its local blocks. For SEIR with one million cells, the bytes sent per rank halve with each doubling of rank count, while the measured gathered receive volume remains approximately constant at 5.24 GB per rank. As the rank count increases, this receive volume is paired with progressively less local computation, reducing the opportunity to hide communication.

This pattern is consistent with the efficiency decline beyond eight ranks in both communication modes. The decline is more pronounced for TIV, whose per-cell computational cost is approximately six times lower than SEIR's: TIV reaches only 22–32% efficiency at 32 ranks. Thus, overlap mitigates part of the exposed communication cost, while the replicated history exchange remains a scalability constraint, particularly for less computationally intensive models.

D. Within-Node MPI Rank Decomposition

Multiple MPI ranks also improve performance within a single node. On the Threadripper system, partitioning the same 64 hardware threads into two ranks with 32 threads each or four ranks with 16 threads each outperforms a single 64-thread rank in 71 of 72 strong- and weak-scaling configurations. The four-rank, 16-thread layout is typically the fastest, reducing runtime by 8–10% for HER, 1–5% for SEIR, and 16–29% for TIV relative to the single-rank configuration.

These results suggest that rank decomposition improves the organization of shared-memory execution. Smaller per-rank thread pools may reduce scheduling and synchronization contention, while smaller block and history working sets may improve cache locality. These mechanisms are plausible explanations for the measured improvements, rather than individually isolated causes.

With the shared-memory MPI transport, overlapped and immediate-wait exchanges differ in runtime by at most 5.5%, with no consistent winner. The principal benefit in this setting therefore comes from rank decomposition rather than communication overlap.